

Space Colony: algorithms.

Nicolás Acero; Erik Fernández; Laura Paez.
'Universidad Distrital Francisco José de Caldas'

Abstract

Space Colony is a fully implemented turn-based strategy game played on an 8×8 board, where a human player competes against an artificial intelligence (AI) to conquer the planet. The AI combines greedy algorithms for expansion and attack decisions with a backtracking algorithm for planning resource-maximizing routes under energy and distance constraints. The system integrates a C++ game engine, custom data structures, and a Python/Pygame graphical interface connected through JSON communication. Testing demonstrated the correct operation of the algorithms, successful integration of all software components, and stable gameplay performance.



Introduction

Space Colony is a turn-based strategy game designed to apply algorithms and data structures in a practical environment. In the game, a human player competes against an artificial intelligence (AI) to colonize a planet while managing energy and distance constraints.

The AI uses greedy algorithms for expansion and attack decisions, and a backtracking algorithm to plan resource-maximizing routes. The system combines a C++ game engine, custom data structures, a Pygame graphical interface, and JSON-based communication between software components.

Methods and Materials

- **C++ Engine:** Implements the game logic using a linked list for colony history and an AVL tree for efficient colony management.
- **AI Algorithms:** Greedy algorithms handle expansion and attack decisions, while backtracking optimizes resource-collection routes.
- **Pygame Interface:** Displays the game board, HUD, and player interactions.
- **JSON Communication:** Synchronizes data between the interface and the game engine through *input.json* and *state.json*.

Greedy Algorithms

Greedy Expansion:

Select the adjacent cell that is free with the most resources if the power is sufficient.

It has complexity $O(n)$, n being the cells available.

Greedy Attack:

Selects the adjacent enemy colony with fewer resources (less defense).

It has complexity $O(n)$ being n the adjacent enemies.

Backtracking Algorithms

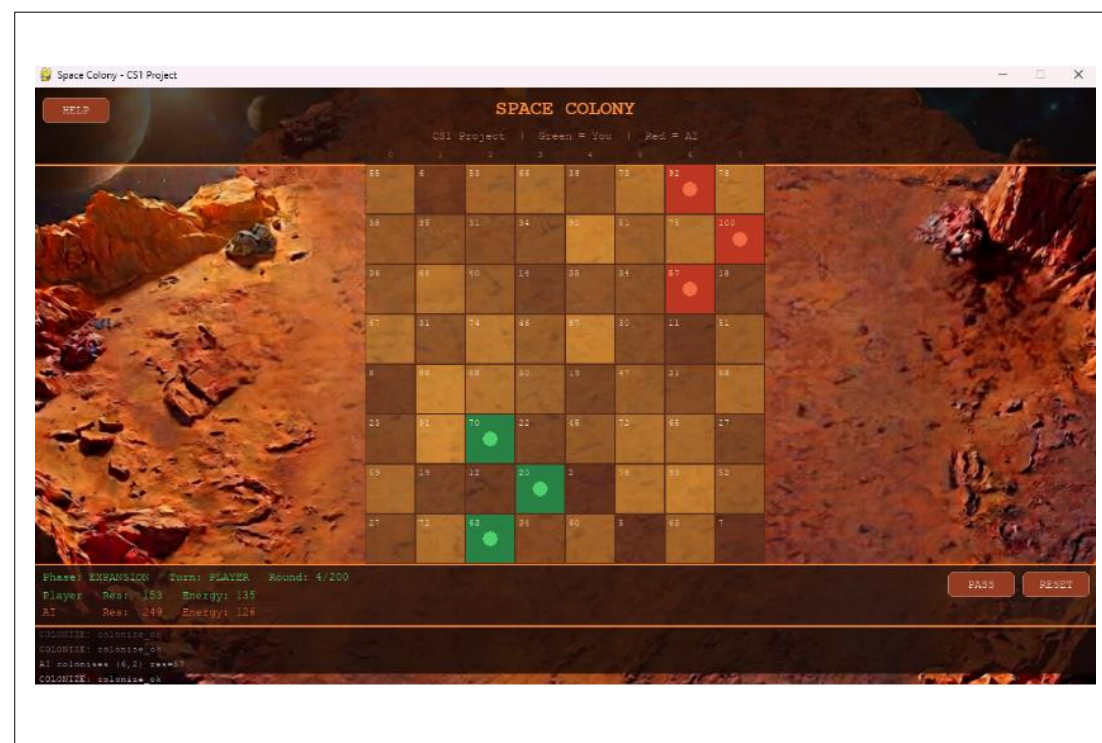
Explore expansion paths that maximize resources, while respecting:

- Remaining Energy
- Minimum distance between colonies

Exponential complexity in the worst case, but with pruning it is feasible in 8×8.

Linked list

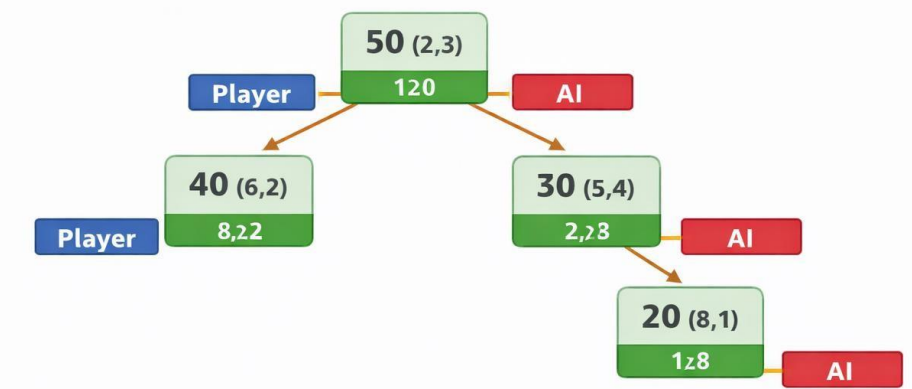
Stores colonies in chronological order, preserving the history of expansions and allowing efficient insertion operations during gameplay.



AVL Tree

Stores colonies ordered by resources and guarantees ($O(\log n)$) operations. It allows the AI to efficiently locate strategic colonies for expansion and attack decisions.

AVL Tree – Resource Deposits



Discussion

The experimental results demonstrate that the implemented algorithms are effective for decision-making during gameplay. The greedy algorithms provide fast responses for expansion and attack actions, while the backtracking algorithm, although computationally more expensive, is capable of finding resource-maximizing routes within the constraints of the Space Colony game. The custom data structures implemented in C++ operated correctly throughout testing, and the AVL tree ensured efficient search and update operations.

The communication mechanism between the graphical interface and the game engine proved reliable, maintaining low latency and consistent synchronization through JSON files. Integration tests confirmed that game states were correctly exchanged between software components.

Conclusions

A competitive strategy game was successfully developed in which an artificial intelligence uses greedy and backtracking algorithms to make expansion and attack decisions. The project demonstrates the practical application of algorithmic techniques in a video game environment, allowing the AI to optimize resource collection and territory control.

Testing confirmed the correctness of the implemented algorithms, the stability of the software architecture, and the functionality of the complete system. Overall, the project met its objectives and provided a functional and engaging gameplay experience. Future extensions may include additional AI strategies, larger game maps, multiplayer support, and further optimization of the decision-making algorithms.